

RB-OPS-001：libsql异常连接问题处理Runbook

Metadata

- 环境: 赋能
- 服务名称: distapp
- 告警名称: 连接异常
- 负责人: @sre-oncall
- 最后更新: 2026-07-20
- 下次演练: xxx
- 预计耗时: 5-20 分钟
- 严重级别: P1 (全量不可用)

触发条件

- ☒ 用户反馈：Web页面无法正常显示

前置检查清单

- ☒ 已连jumpserver
- ☒ 进入docker部署的dist服务192.168.13.164
- ☒ 已通知值班群：「<service-name> 系统繁忙故障排查中，@oncall」

快速诊断

步骤1：查看web日志

```
# docker logs -f distapp-distapp-1

# 发现报错
_ _ _ | | _ | |
/ _|/ _` | |/_` |
\_ \ (| | | (| |
|_/_\_, |/_\_,-|
      |_|

welcome to sqld!

version: 0.24.31
commit SHA: e88c6b513da5e87e2051c8dd58797ef367392440
build date: 2025-01-06

This software is in BETA version.
If you encounter any bug, please open an issue at
https://github.com/tursodatabase/libsql/issues
```

```

config:
  - mode: primary (0.0.0.0:5001)
  - database path: iku.db
  - extensions path: <disabled>
2026-07-20T02:26:22.987909Z INFO sqld: listening for incoming user HTTP
connection on 0.0.0.0:8080
2026-07-20T02:26:22.987924Z INFO sqld: Using legacy HTTP basic authentication
2026-07-20T02:26:22.987949Z INFO sqld: listening for incoming gRPC connection on
0.0.0.0:5001
  - listening for HTTP requests on: 0.0.0.0:8080
  - grpc_tls: no
2026-07-20T02:26:22.989883Z INFO restore: libsql_server::namespace::meta_store:
restoring meta store
2026-07-20T02:26:22.989950Z INFO restore: libsql_server::namespace::meta_store:
meta store restore completed
2026-07-20T02:26:22.990618Z INFO libsql_server: Server sending heartbeat to URL
<not supplied> every 30s
2026-07-20T02:26:22.991189Z INFO libsql_server::rpc: serving internal rpc server
without tls
2026-07-20T02:26:22.993673Z INFO
create:try_new_primary:make_primary_connection_maker:
libsql_server::replication::primary::logger: Replication log is dirty, recovering
from database file.
2026-07-20T02:26:22.995248Z INFO
create:try_new_primary:make_primary_connection_maker:
libsql_server::replication::primary::logger: SQLite autocheckpoint: 1000
Error: Internal Error: `EOF while parsing a value at line 1 column 0`

Caused by:
  EOF while parsing a value at line 1 column 0

```

步骤2：查看libsql数据存放位置

```
[10:03:53 root@DistApp distapp]#vim docker-compose.yml
```

```

services:
  distapp:
    image: 5yunus2efendi/distapp:latest
    platform: linux/amd64
    ports:
      - "3000:3000"
    env_file:
      - docker-compose-base.env
      - docker-compose.env
    depends_on:
      - db
  db:
    image: ghcr.io/tursodatabase/libsql-server:v0.24.31
    platform: linux/amd64
    env_file:
      - docker-compose.env
    volumes:

```

```
- db:/var/lib/sqlld
volumes:
  db: # 使用命名卷
```

步骤3: 查看docker数据存放位置

```
systemctl cat docker
...
[Service]
Type=notify
# the default is not to use systemd for cgroups because the delegate issues still
# exists and systemd currently does not support the cgroup feature set required
# for containers run by docker
ExecStart=/usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
--data-root=/opt/data/docker # 在docker数据在此处
ExecReload=/bin/kill -s HUP $MAINPID
TimeoutStartSec=0
RestartSec=2
Restart=always

# Having non-zero Limit*s causes performance problems due to accounting overhead
# in the kernel. We recommend using cgroups to do container-local accounting.
LimitNPROC=infinity
LimitCORE=infinity

# Comment TasksMax if your systemd version does not support it.
# Only systemd 226 and above support this option.
TasksMax=infinity

# set delegate yes so that systemd does not reset the cgroups of docker
containers
Delegate=yes

# kill only the docker process, not all processes in the cgroup
KillMode=process
OOMScoreAdjust=-500

[Install]
WantedBy=multi-user.target
```

步骤4: 查看libsql卷存储

```
#tree /opt/data/docker/volumes/distapp_db/_data/iku.db/
/opt/data/docker/volumes/distapp_db/_data/iku.db/
├── dbs
│   └── default
│       ├── data
│       ├── data-shm
│       └── data-wal
```

```
|      |— stats.json
|      |— tmp
|      |— to_compact
|      |— wallog
|      |
|      └─ metastore
|          |— data
|          |— data-shm
|          └─ data-wal
```

步骤5：结合web服务报错分析解决

这个错误 `EOF while parsing a value at line 1 column 0` 是 Rust `serde_json` 在解析空文件时的典型报错。结合目录结构，问题几乎可以确定出在：

```
dbs/default/stats.json
```

根因分析：

日志显示：

1. `Replication log is dirty, recovering from database file.` — 上次异常关闭导致 WAL/replication 日志损坏
2. 恢复过程中 `libsql-server` 尝试读取 `stats.json`
3. 该文件可能在上次崩溃后变成了 **0 字节空文件**
4. `serde_json` 解析空文件 → 抛出 `EOF while parsing a value at line 1 column 0`

注意：操作前务必做好备份。`data` 文件是你的实际 SQLite 数据库，只要它不损坏，数据就是安全的。

解决方案1：修复 stats.json（推荐，数据无损）

先检查该文件是否为空：

```
ls -la /opt/data/docker/volumes/distapp_db/_data/iku.db/dbs/default/stats.json
cat /opt/data/docker/volumes/distapp_db/_data/iku.db/dbs/default/stats.json
```

```
file /opt/data/docker/volumes/distapp_db/_data/iku.db/dbs/default/stats.json
wc -c /opt/data/docker/volumes/distapp_db/_data/iku.db/dbs/default/stats.json
```

如果为空或损坏，写入一个最小合法的 JSON 对象：

```
echo '{}' >
/opt/data/docker/volumes/distapp_db/_data/iku.db/dbs/default/stats.json
```

重启容器

```
docker restart libsql
```

解决方案二：清理恢复（如果方案一无效）

如果 `stats.json` 修复后仍然报错，可能是 `wallog` 目录或 `metastore` 也损坏了。可以按顺序尝试：

```
cd /opt/data/docker/volumes/distapp_db/_data/iku.db/

# 1. 备份当前数据
cp -r dbs/default dbs/default.bak.$(date +%s)

# 2. 删除可能损坏的 JSON 和临时文件
rm -f dbs/default/stats.json
rm -rf dbs/default/tmp/*
rm -rf dbs/default/to_compact/*

# 3. 重建 stats.json
echo '{}' > dbs/default/stats.json

# 4. 重启容器
docker restart libsql
```

解决方案三：极端情况 — 从 data 文件重建

如果上述都无效，说明 WAL 日志和 metadata 已严重损坏，但 SQLite 的 `data` 文件通常还是完整的。可以：

```
# 1. 停止容器
docker stop libsql

# 2. 备份整个目录
cp -r /opt/data/docker/volumes/distapp_db/_data/iku.db
/opt/data/docker/volumes/distapp_db/_data/iku.db.bak.$(date +%s)

# 3. 保留 data 文件，删除其他所有元数据
cd /opt/data/docker/volumes/distapp_db/_data/iku.db/dbs/default
mv data /tmp/iku_data_backup
rm -rf *
mkdir -p tmp to_compact wallog
mv /tmp/iku_data_backup data
echo '{}' > stats.json

# 4. 对 metastore 同样处理
cd /opt/data/docker/volumes/distapp_db/_data/iku.db/metastore
rm -f data-shm data-wal

# 5. 启动容器，让它自动重建元数据
docker start libsql
```